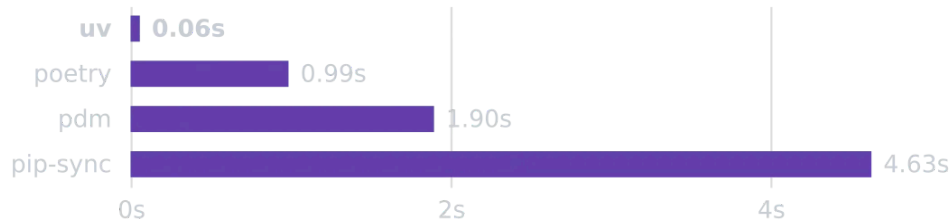


How to Install UV package manager on Windows, Linux, Mac, and Docker

 vsuhas.medium.com/uv-package-manager-180cc63c3b18

Suhas Vaddhiparthi

June 6, 2025



uv: An extremely fast python package & projectmanager (image from uv docs)

The Python ecosystem is constantly evolving, and package managers play a crucial role in managing dependencies. Enter uv, a blazing-fast Python package manager by Astral that aims to be a drop-in replacement for tools like pip, pip-tools, and virtualenv.

Join Medium for free to get updates from this writer.

Remember me for faster sign in

In this guide, I'll walk you through setting up uv on **Windows**, **Linux**, and **macOS**, and show you how to use it effectively.

What is UV 🔍?

UV is an all-in-one Python package manager written in Rust. It combines multiple tools into one.

1. Fast dependency resolution and installation ⚡
2. Lockfile support (uv pip compile) 🔒
3. Virtual environment management 📦
4. Full compatibility with pyproject.toml and requirements.txt ✅

UV is currently supported in python 3.8 or greater versions.

Press enter or click to view image in full size



Photo by [Johannes Plenio](#) on [Unsplash](#)

Installing UV Package Manager:

1. Windows OS 🪟:

```
# Open Windows Power Shell and Run the below command
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

```
# Using winget
winget install --id=astral-sh.uv -e
```

```
# Using scoop
scoop install main/uv
```

2. Linux OS 🐧:

```
# Using curl
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# using wget
wget -qO- https://astral.sh/uv/install.sh | sh
```

```
# Add to path
export PATH="$HOME/.cargo/bin:$PATH"
```

3. Mac OS 🍏:

```
# Using Homebrew
brew install uv
```

```
# Using curl
curl -Lsf https://astral.sh/uv/install.sh | sh
```

```
# using wget
wget -qO- https://astral.sh/uv/install.sh | sh
```

```
# Add to path
export PATH="$HOME/.cargo/bin:$PATH"
```

4. Docker Container 🐳:

```
# Creates a Python Virtual ENV as well
```

```
FROM nvidia/cuda:12.2.0-devel-ubuntu22.04
```

```
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
```

```
# Create a .venv virtual environment using python 3.12
# Note: The container need not have python3
RUN uv venv --python 3.12
```

```
...
```

5. Other Options:

```
# Using cargo (builds first)
cargo install --git https://github.com/astral-sh/uv uv
```

```
# pip
pip install uv
```

Running a Python Script using UV 📁:

```
# Note: Need not activate the virtual environment
uv run main.py
```

```
# Unzipping using python3
uv run -m zipfile -c zip_file_name.zip .
```

```
# The above command is equivalent to
source .venv/bin/activate
python3 -m zipfile -c zip_file_name.zip .
```

Setting up project and Virtual Environment :

1. Setting up a project

```
# Initializing a project called "app"
# This will create:
# 1. main.py file with basic print statement
# 2. .python-version file
# 3. pyproject.toml contains the project metadata
# 4. Readme.md. file
```

```
uv init app
```

```
# Giving the app name is not mandatory
uv init
```

2. Managing Packages without using pip

```
# Add a dependency
# Note: Autotmatically creates a .venv virtual environment folder
uv add numpy
```

```
# Remove a dependency
uv remove numpy
```

```
# Sync the project's dependencies with the env.
uv sync
```

```
# Create a lockfile for the project's dependencies.
# Note: creates a uv.lock file
uv lock
```

```
# Show the package dependency tree
uv tree
```

3. Managing Packages using pip

```
# Add a dependency
# Note: Automatically creates a .venv virtual environment folder
uv pip install numpy
uv pip install -r requirement.txt
```

```
## Install from .toml file as well as numpy
uv pip install -r pyproject.toml numpy
```

```
# Remove a dependency
uv pip uninstall numpy
```

```
# Sync the project's dependencies with the env.
uv pip sync
```

```
# Show the list of installed packages
uv pip show
```

4. Manual Virtual Environment Creation

```
# Creates a .venv with latest python version
uv venv
```

```
# Create a .venv with specific python version
uv venv --python 3.12
```

Running a Python Script using uv

```
# Note: Need not activate the virtual environment
uv run main.py
```

```
# Unzipping using python3
uv run -m zipfile -c zip_file_name.zip .
```

```
# The above command is equivalent to
source .venv/bin/activate
python3 -m zipfile -c zip_file_name.zip .
```

Conclusion:

If you're looking for a modern, Rust-powered Python package manager that's fast, compatible, and easy to use: uv is the way to go. It simplifies everything from virtual

environment management to lock-file generation, while remaining compatible with Python standards.

References:

1. <https://docs.astral.sh/uv/>